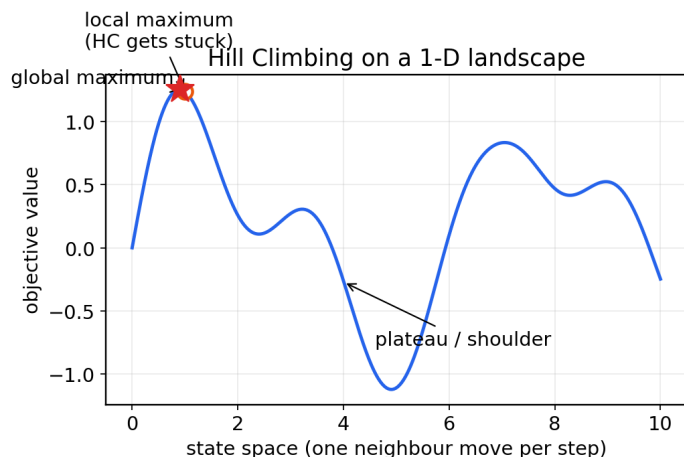


Core Idea

Keep **one current state** (or k), step to a better *neighbour*. The **solution is the state itself**, not the path. Memory-cheap; no optimality guarantee. Three ingredients: state space $\mathcal{S}$, neighbourhood $N(s)$, objective f (minimise $h \equiv$ maximise $-h$).

The Landscape

Global max (true goal) · **local max** (HC stops here) · **plateau** (flat, no improving move) · **shoulder** (plateau then rises) · **ridge** (narrow, slides off). All algorithms differ in how aggressively they accept *non-improving* moves.



Hill Climbing Variants

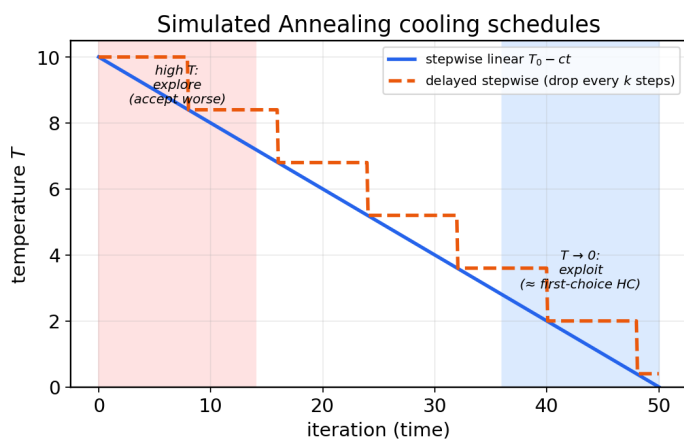
- **Steepest-ascent**: move to best neighbour; stops at first local optimum.
- **Stochastic**: random *improving* neighbour (strict $>$). Does **not** escape plateaus.
- **First-choice**: take first improving neighbour found — best for *huge* neighbourhoods.
- **Random-restart**: many random seeds, keep best. $P(\text{global}) \rightarrow 1$ as restarts $\rightarrow \infty$ (no finite guarantee).

Simulated Annealing

Pick a **random** neighbour; always accept if better, else accept worse with

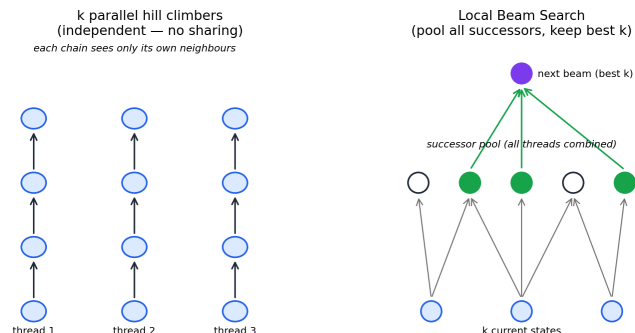
$$P(\text{accept}) = e^{\Delta E/T}, \quad \Delta E = f(s') - f(s) < 0.$$

High $T \Rightarrow$ explore (random walk); $T \rightarrow 0 \Rightarrow$ **first-choice HC** (not steepest!). Cooling (lecture): *stepwise linear* $T_0 - ct$ and *delayed stepwise*. Slower cooling = better quality, more time; no finite-time guarantee.



Local Beam Search

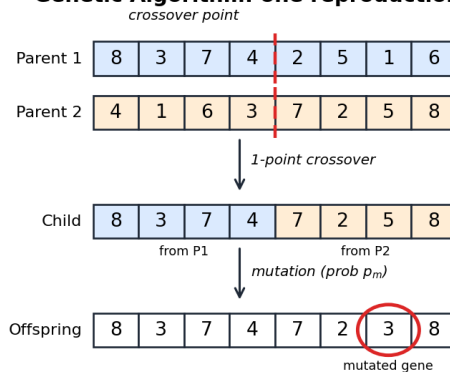
Keep k states; generate *all* successors of all k ; keep the **best k overall**. $\neq k$ **parallel hill climbers** — the beam *shares information*, so a good region attracts all threads (risk: premature convergence). Stochastic beam $\Rightarrow$ diversity.



Genetic Algorithms

Population of **chromosomes** (strings of genes); **fitness** = objective. **Selection** fitness-proportionate $\rightarrow$ **1-point crossover** $\rightarrow$ **mutation** (prob. p_m). Crossover combines building blocks (“teleports” across landscape). **Permutations (TSP)**: naive crossover breaks validity $\Rightarrow$ use *uniform-order crossover* (+ swap mutation).

Genetic Algorithm: one reproduction step



Selection is fitness-proportionate: fitter parents are picked more often.

Comparison

	Down?	Mem	Escapes?
Steepest HC	no	1	no
Stochastic HC	no	1	no
First-choice HC	no	1	no
Random-restart	no	1	via restarts
Sim. Annealing	yes (prob)	$1+T$	yes
Local Beam (k)	no	k	limited
Genetic Alg.	indirect	pop	yes

Exam Traps

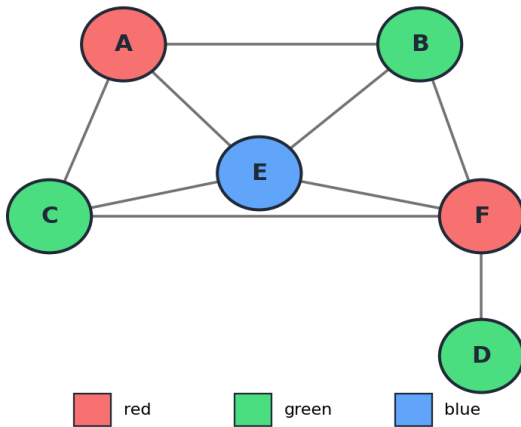
- SA exponent sign: $\Delta E < 0 \Rightarrow e^{\Delta E/T} \in (0, 1)$. Never $e^{|\Delta E|/T}$.
- $T \rightarrow 0$: SA = **first-choice**, not steepest.
- Stochastic HC needs a *strictly* improving move $\Rightarrow$ stuck on plateaus.
- Beam search $\neq$ parallel HC (information sharing).
- Naive crossover on permutations $\Rightarrow$ invalid tours.
- HC = discrete gradient ascent; both trapped by local optima.

Formalism

A CSP is a triple $(\{X_i\}, \{D_i\}, R)$: **variables**, **domains**, **constraints** (relations). A **solution** = *complete* (all assigned) + *consistent* (no constraint violated). *Normalised* CSP = only unary/binary constraints. Implicit $(X_1 < X_2)$ beats explicit (list of tuples) for cheap tests.

Constraint graph: node = variable, edge = binary constraint. Tree-structured graphs solve in *linear* time $O(nd^2)$.

Map-colouring constraint graph
(nodes = variables, edges = $\neq$ constraints)

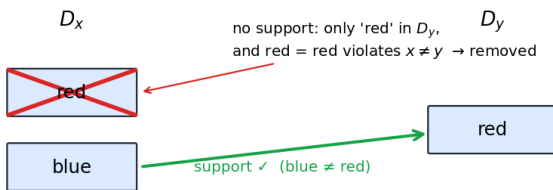


Consistency (preprocessing)

Consistency = **no constraint violated**. Variants are the same idea at different scales:

- **Node (1-consistency)**: every value satisfies the unary constraint. Trivial.
- **Arc (2-consistency)**: $\forall x \in D_i \exists y \in D_j : P_{ij}(x, y)$. **Directional** — arc $(X_i, X_j) \neq (X_j, X_i)$.
- **Path (3-consistency)**, k -consistency: strong n -consistency $\Rightarrow$ no backtracking (but exponential to enforce).

Arc consistency — Revise(x, y) under constraint $x \neq y$



AC-3

Queue of all directed arcs. Pop (X_i, X_j) , REVISE (delete $x \in D_i$ with no support in D_j); if D_i shrank, **re-add every arc** (X_k, X_i) . Empty domain $\Rightarrow$ INCONSISTENT.

Complexity $O(cd^3)$, $c = \#$ directed arcs, $d = \max |D_i|$.

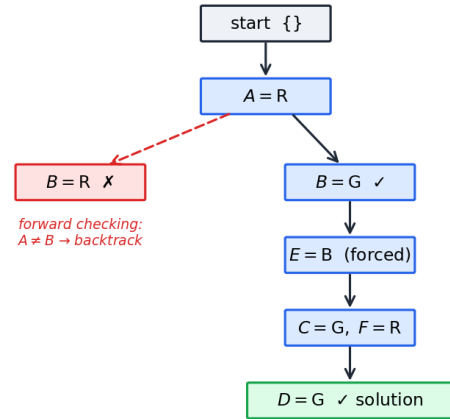
$c \cdot d$ re-queues $\times O(d^2)$ per revise. **Non-empty domains do NOT prove a solution exists** (AC only *disproves*).

Backtracking Search

Assign one var at a time, check consistency, backtrack on violation. *Complete* algorithm. Guided by heuristics:

- **MRV** (var order, *fail-first*): pick smallest remaining domain.
- **Degree** (tie-break): most constraints on unassigned vars.
- **LCV** (value order, *succeed*): value ruling out fewest neighbour options.
- **Forward checking**: prune neighbours' domains after each assignment (1 step). **MAC** = full AC-3 after each assignment (prunes more).

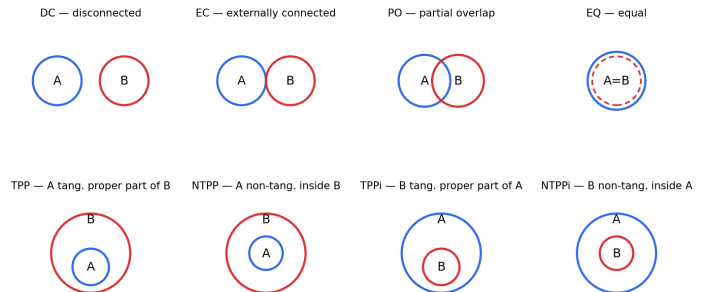
Backtracking search with MRV + forward checking



Spatial Reasoning: RCC-8

8 mutually-exclusive, jointly-exhaustive relations between two regions. i = inverse (swap A, B); TPP vs NTPP differ only in touching the boundary.

RCC-8: eight base relations between two regions



$DC \cdot EC \cdot PO \cdot EQ \cdot TPP \cdot NTPP \cdot TPPI \cdot NTPPi$.

Local Search for CSPs

Min-conflicts: from a complete (invalid) assignment, repeatedly reassign a conflicted variable to minimise conflicts. Fast, scales (large n -queens) but *incomplete* — cannot certify inconsistency.

Exam Traps

- Arc consistency is **directional** — revise both ways; re-add incoming arcs.
- $O(cd^3)$: c counts *directed arcs* = $2 \times$ binary constraints.
- Non-empty domains after AC $\neq$ solvable (3 vars, 2 colours, all $\neq$).
- **MRV** picks the *variable*; **LCV** picks the *value* (opposite goals).
- **Two senses of “complete”**: *assignment* (all vars assigned) vs. *algorithm* (finds a solution if one exists). Backtracking = partial assignments, complete *algorithm*. Local search = complete *assignments* (possibly invalid), *incomplete* algorithm.